

L-PCN: A Point Cloud Accelerator Exploiting Spatial Locality through Octree-based Islandization

Yiming Gao^{†‡}, Jieming Yin[†], Yuxiang Wang[†], Xiangru Chen[‡],
Zhilei Chai[§], Bowen Jiang[§], Jiliang Zhang^{†*}, and Herman Lam[‡]

[†] Nanjing University of Posts and Telecommunications [‡]University of Florida [§]Jiangnan University
{gaoyiming, hlam}@ufl.edu {jieming.yin, zhangjiliang}@njupt.edu.cn

Abstract—Existing Point Cloud Networks (PCNs) have proven to achieve great success in many point cloud tasks such as object part segmentation, shape classification, and so on. The most popular point-based PCNs are usually composed of two sequential steps: Data Structuring (DS) and Feature Computation (FC). In this paper, we first describe an important characteristic of the PCN-specific DS step that has not been addressed in existing PCN accelerators: the spatial locality resulting from overlapping points of the gathered point subsets. Using algorithm-hardware co-design, L-PCN (Locality-aware PCN) proposes two novel techniques to exploit this characteristic to reduce the large amount of repetitive operations in the overall PCN. The first of which is a point cloud partitioning technique, Octree-based Islandization. Using Octree-based adjacency gathering, a point cloud is partitioned into islands in L-PCN, where the point subsets inside the same island exhibit a strong spatial correlation. After partitioning, L-PCN performs the rest of PCN steps at the granularity of islands. The second method of L-PCN is scheduling the intra-island computation with a Hub-based Scheduling to exploit the intra-island data reuse by dynamically caching, updating, and reusing the repeated data. The two methods are implemented in an Islandization Unit, which can be seamlessly integrated into standard PCN workflow. Our evaluation shows that based on our methods for exploiting spatial locality, L-PCN achieves a theoretical reduction in feature fetching ranging from 55.2% to 93.8% and in feature computation ranging from 45.4% to 80.6% during the PCN process. For experimentation, prototype L-PCN accelerators are implemented on the Intel Arria 10 GX FPGA. Experimental results prove that with the Islandization Unit as a plug-in, state-of-the-art PCN accelerators can achieve an additional speedup ranging from $1.2\times$ to $3.2\times$.

I. INTRODUCTION

With the development of 3D sensors, such as LiDARs and RGB-D cameras, point cloud data has become prevalent in modern embedded applications such as autonomous driving [32], robotics [25], AR/VR [44], [49]. Point Cloud Networks (PCNs) have proven to achieve great success in different tasks of point cloud analysis, including object part segmentation [4], shape classification [45], and semantic segmentation [2], [11].

Current PCNs can be classified into the following categories according to the method used [22]: 2D-CNN based [8], [27], [52], 3D-CNN based [21], [28], [42], and point-based [39], [47], [48]. Among these categories of PCNs, the point-based PCNs are most widely used and their basic building block can be generalized into an architecture comprising two steps: Data Structuring and Feature Computation, as shown in Figure 1. In

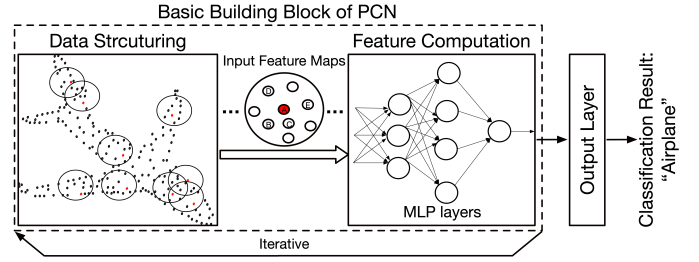


Fig. 1. Standard workflow and the two steps in the basic building block of point-based PCN: Data Structuring and Feature Computation.

a point-based PCN (will subsequently be referred to as PCN for simplicity), the Data Structuring (DS) step is to gather the localized points to form point subsets. For example, the points in each black circle in Figure 1 are gathered to form a point subset. Together, these point subsets are the input feature maps for the Feature Computation (FC) step. The features of the point subsets serve as inputs for the MVM (Matrix-Vector Multiplication) operations needed for the convolutional layers of the Feature Computation step.

In a general-purpose PCN architecture in GPU and CPU platforms, both the Data Structuring and Feature Computation steps consume a non-trivial part of overall latency [31]. With existing domain-specific designs [16], [31], the Feature Computation step performs the same functions as traditional Deep Neural Networks (DNNs), and can be directly accelerated by existing commercially available Deep Learning Accelerators (DLAs) such as NPUs [13]. However, the Data Structuring step is a PCN-specific operation and cannot be directly supported by these commercially available DLAs. If unaccelerated, the Data Structuring step will become a major bottleneck in PCN tasks, as demonstrated in [16], [31], [53].

Domain-specific PCN accelerators primarily target the DS step by developing customized units, while delegating the FC step to DLAs (e.g., NPUs). They can be categorized into two types: **Accurate DS** accelerators [19], [31], which perform accurate neighbor search as in traditional methods; **Approximate DS** accelerators [14], [53], which adopt approximate neighbor search for higher DS efficiency. With these efforts reducing DS overhead (e.g., to less than 15% in EdgePC [53]), the FC step (typically optimized only by DLAs) has in turn become the major source of latency in current PCN accelerators (e.g., up to above 85% in [14], [19], [31], [53]).

*Corresponding author.

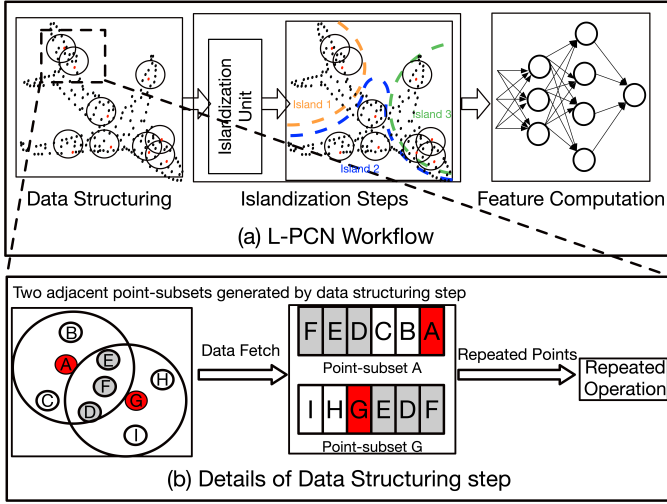


Fig. 2. (a) L-PCN workflow: adding Islandization Steps to exploit spatial locality. (b) Details of Data Structuring step, illustrating the overlap of points between point subsets.

As will be discussed in more detail in Section II, there is an important characteristic of the Data Structuring step that has not been addressed in existing PCN accelerators: the spatial locality resulting from overlapping points of the gathered point subsets, which can be exploited to reduce the large amount of repetitive operations in the overall PCN.

Many existing CNN and GCN (Graph Convolutional Network) accelerators [7], [10], [20] have effectively leveraged data locality from shared neighbors among adjacent input feature maps to eliminate redundant operations. In this work, we demonstrate that, in a similar manner, data redundancy also exists in PCNs in a spatially overlapping manner. As illustrated by the left part of the L-PCN workflow in Figure 2(a), the PCN-specific Data Structuring step first picks a set of central points (red points) and gathers its neighboring points to form point subsets (points within the black circles). Figure 2(b) provides some details of such a point subset. Points A and G are selected as central points of the two example point subsets. The grey points (D, E, F) are parts of both point subsets (i.e., overlap). From a benchmarking study that will be discussed in Section II-B, up to above 90% overlap of gathered points (grey points in Figure 2(b)) can be observed among adjacent point subsets. Consequently, this data redundancy leads to non-trivial repeated memory accesses and computations in PCNs. However, unlike GCNs and CNNs, PCNs face unique challenges for exploiting spatial locality due to the spatial sparsity of point clouds and the non-adjacent execution order in standard workflows, as further discussed in Section III-A.

To effectively exploit spatial locality in PCNs, L-PCN enhances the PCN workflow by incorporating novel Islandization steps (between Data Structuring and Feature Computation), implemented through an Islandization Unit. The L-PCN workflow, shown in Figure 2(a), will be detailed in Section III, and is briefly described here. To discover the overlapping points between point subsets (gathered by Data Structuring step), L-PCN uses a novel point cloud segmentation method, **Octree-**

based Islandization, to partition the point cloud by clustering adjacent point subsets into L-PCN Islands. L-PCN Islands are groups of point subsets with strong spatial correlation, i.e., sharing a large number of overlapping points. As the example shown in the middle Figure 2(a), the Islandization Unit partitions the example point cloud into Island 1 (orange), Island 2 (blue), and Island 3 (green). A key feature of L-PCN is that a point subset cannot belong to more than one island.

After the Octree-based Islandization process, L-PCN performs the remaining PCN steps at the granularity of each island, instead of the granulation of each point subset as in existing PCN accelerators. To exploit the intra-island data reuse, we propose an **Hub-based Scheduling** method to schedule [37] the intra-island computation, resulting in high data locality during the runtime processing of an Island. This method deploys a novel cache design to reuse overlapping data, thereby avoiding redundant memory fetch operations after the Data Structuring step. It also results in avoiding repeated computation during the Feature Computation step related to these overlapping points. As shown in Figure 2(a), these two methods—Octree-based Islandization and Hub-based Scheduling—are implemented in the Islandization Unit. Both methods rely on Octree search [33] as the basic operation and performed using an Octree-search Engine. The detailed strategies and supported architectures of these two methods will be detailed in Section IV and Section V.

The L-PCN prototype, along with the evaluation, is described in Section VI. The evaluation is performed on two representative PCN models: PointNet++ and DGCNN, as well as two large-scale PCN variants: PointNeXt and PointVector, using four modern point cloud datasets. The experiments show that, in terms of theoretical workload optimization, the Islandization-based methods (for exploiting spatial locality) reduce memory fetching and computation workloads, ranging from 55.2% to 93.8% and from 45.4% to 80.6%, respectively. The prototype L-PCN accelerators are implemented on the Intel Arria 10 GX FPGA. We further show that L-PCN’s Islandization Unit can be integrated into both categories of existing PCN accelerators (i.e., accurate and approximate) as a plug-in, enhancing their performance with an additional speedup from $1.2\times$ to $3.2\times$. Moreover, L-PCN’s methods for exploiting spatial locality outperform the workload reduction mechanism of the state-of-the-art PCN accelerators, GDPCA [5] and Mesorasi [16].

In Section VIII, we provide the conclusions and discuss the broader applications of our work.

II. BACKGROUND AND MOTIVATION

In this section, we present the background and motivation for L-PCN. Sub-section II-A introduces some of the key PCN basics. Sub-section II-B explains our motivation for exploiting spatial locality caused by shared neighbors using a representative PCNs, PointNet++ [39], as an example.

A. PCN Basics: General Operations of PCN Steps

Figure 1 illustrates the general steps followed by current PCNs in processing a basic PCN Building Block: a Data Structuring (DS) step, followed by a Feature Computation (FC) step. For example, in PointNet++ [39], such a PCN Building Block is called a Set Abstraction; in DGCNN [48], it is called EdgeCONV. As shown in Figure 1, a PCN process needs to iterate through several such PCN Building Blocks before processing the output layer. Figure 3 illustrates the general operations within these two steps.

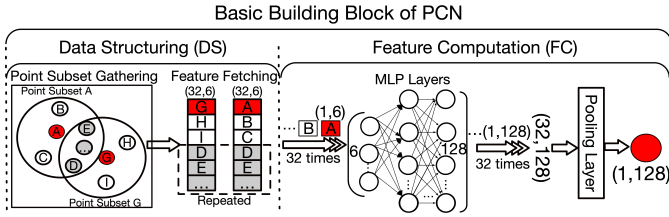


Fig. 3. Details of the two steps in a basic PCN Building Block

The Data Structuring (DS) step is used to adapt the spatial sparsity of the points in point cloud to form the “input feature map” for the Feature Computation (FC) step. The FC step is the actual convolutional step typically implemented using MLP layers. Of these two types of operations, DS is unique to PCN. The DS step first picks a set of points as central points, then performs the neighbor-point gathering around these central points to form the point subsets. The feature information of these point subsets, akin to the input feature map in traditional DNNs, are sequentially inputted into the MLP layers for feature computation.

A point cloud comprises a collection of points denoted as $x = \{(p_n, f_n)\}$, where each $p_n = (x_n, y_n, z_n)$ is the 3D coordinate of a point, and f_n is its feature vector. The DS step forms the point subsets by leveraging the spatial coordinates $p_n = (x_n, y_n, z_n)$ to perform neighbor gathering. Each central point picks the K spatially nearest points to form the point subset through a neighbor gathering algorithm, like KNN (K-Nearest Neighbors) [48] or BQ (Ball Query) [39]. In Figures 2 and 3, the points enclosed by a black circle represent a formed point subset resulting from the DS step. The K (enclosed point number of each black circle) is a fixed number (e.g., $K = 32$), as the example shown in Figure 3. For the rest of this section, we will use $K = 32$ for illustration.

After the point subsets are formed, the 1-D feature vectors f of the points included in each point subset need to be retrieved from memory to create input feature maps. Within an input feature map, the feature of each point enters the MLP layers one by one. The dimension of the feature vector f will be transformed during the MLP layers. As shown in Figure 3, during the processing of the MLP layers, the dimension of feature f of each point will be changed from (1, 6) to (1, 128), resulting in the entire point subset being transformed from (32, 6) to (32, 128). At the final stage of Feature Computation, a Pooling Layer (e.g., max pooling) aggregates the MLP results of the 32-point subset into the central point. *Between*

two 32-points point subsets, if there are N overlapping points, then the feature fetch and MLP computation related to these N points are repeated. In the subsequent Section II-B, we will show the ratio $N/32$ can be quite large, resulting in a high percentage of feature fetch and computation being repeated.

B. Motivation: Exploiting Spatial Locality in PCN

As discussed earlier, similar to CNNs and GCNs, PCNs also adopt a neighbor aggregation scheme, where feature maps are collected from neighboring elements and aggregated into a central element by applying weights. Many existing accelerators for CNNs [10], [26] and GCNs [7], [20], [51] have demonstrated that leveraging data locality from shared neighbors among adjacent feature maps can greatly reduce redundant operations for CNN and GCN processing.

However, existing PCN accelerators have not yet exploited the data redundancy caused by overlapping points between neighbor point subsets. We will now provide a benchmarking study quantifying the overlap-driven optimization opportunity, which motivates our Islandization-based methods to exploit spatial locality and enhance the efficiency of PCN acceleration. The PointNet++ [39] for ModelNet40 [45] dataset will be used as the benchmark for this evaluation.

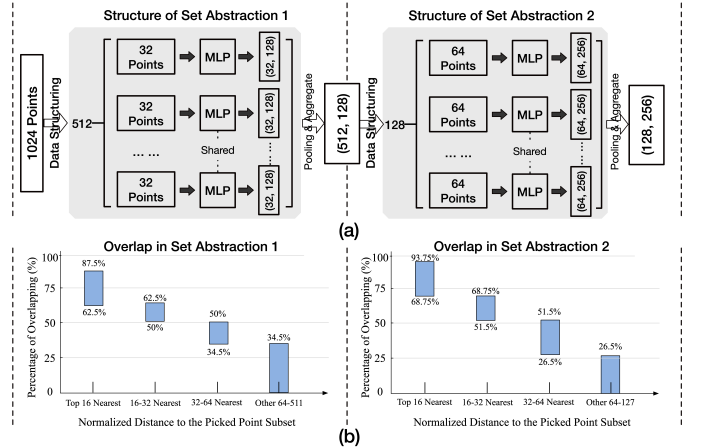


Fig. 4. Detailed breakdown and overlap analysis of two major Set Abstractions (i.e., two iterations of the PCN Building Block) of the benchmark PointNet++ PCN.

Figure 4(a) illustrates the structures of the first two iterations of the Set Abstractions (i.e., Building Blocks) of the benchmark PointNet++ PCN, which collectively consume over 90% of the overall runtime. Figure 4(b) shows the results of an experiment that we performed to validate our hypotheses that the overlapping points (grey points in Figures 2 and 3) are primarily found between neighboring point subsets. And more importantly, among these neighboring subsets, the overlap ratio is very high, providing an opportunity to greatly reduce redundant memory accesses and computations.

In the experiment to generate the results in Figure 4(b), **each point subset** within a Set Abstraction is compared with all other point subsets to determine two items of information:

- 1) The distance between them, which is determined by the distance between their central points.
- 2) The percentage of overlap between the two point subsets. For example, if 17 out of the 32 points of the two point subsets are overlapped, the percentage is 53% (17/32).

In Figure 4(b), the x-axis of each graph is divided into four groups: top 16 point subsets that are closest to a selected point subset, next 16 closest, next 32 closest, and the remainder point subsets which are furthest away. Each of the four bars represents the range of **overlap percentages** for that group.

From the results in Figure 4(b), two observations can be made, which experimentally validate our stated hypotheses. Firstly, overlap of gathered points occurs mostly between point subsets which are spatially adjacent. This is evident because the left-most (nearest) bars of the two Set Abstractions exhibit the highest overlap, whereas the point subsets of right-most (farthest) bars share few or no overlapping points. The second observation is the percentage of overlapping points of neighboring point subsets can be very high; reaching 87.5% (28 out of 32) in Set Abstraction 1 and 93.75% (60 out of 64) in Set Abstraction 2.

III. L-PCN: DESIGN PRINCIPLES AND ARCHITECTURE

A. Design Principles: Detect and Exploit Spatial Locality

The two observations stated at the end of the previous section drive the design principles of the L-PCN’s Islandization methods to detect and cluster adjacent point subsets; and to exploit spatial locality for maximal intra-island data reuse.

Recall from Section II-A (and illustrated in Figure 3), after the point subsets are formed, the feature vectors f of the points included in each point subset need to be retrieved from memory to create input feature maps. Then, during Feature Computation, the feature of each point enters the MLP layers one by one; and MLP layers apply the same weight to every point. As shown in the example in Figure 3, let us assume there are N overlap points between Point-subset A and Point-subset G. During the DS step, without optimization, the feature information f_n of these N points are fetched from memory for Point-subset A and fetched again from memory when processing Point-subset G. Similarly, during the FC step, these N points re-enter the MLP layers for both Point-subset A and Point-subset G.

These redundant operations between the overlapping (usually adjacent) point subsets result in excessive performance cost, but provide a good opportunity to optimize the PCN workload by exploiting spatial locality. However, exploiting spatial locality in PCNs involves unique challenges and is more difficult than in CNNs and GCNs: First, the spatial sparsity of point clouds and the lack of explicit neighbor indexing (e.g., adjacency matrix in graph) make it difficult to detect shared neighbors between point subsets. Moreover, unlike CNNs and GCNs, which process feature maps in a spatially adjacent order (e.g., row-major traversal in CNNs), standard PCN workflows often process point subsets in a spatially distant order. This is because, when forming a new

point subset, PCNs typically use the farthest point sampling (FPS) algorithm to select a central point that is farthest from those already selected. Consequently, the subsets formed and processed in adjacent iterations are often far apart in space.

To address these PCN-specific challenges, the L-PCN’s approach is to gather the adjacent point subsets to detect the overlapping points, and schedule the sequential execution of the PCN steps for these highly overlapping point subsets in such a manner that the repeated memory fetches after Data Structuring and repeated Feature Computation can be avoided.

To this end, we develop an **Octree-based Islandization** method to detect spatial locality caused by shared points by partitioning the point cloud and clustering the adjacent point subsets into islands. As we demonstrated in Section II, highly overlapping point subsets tend to be spatially adjacent. Consequently, the point subsets inside the same island share the most overlap. Within a given island, we deploy a **Hub-based Scheduling** method to exploit the inter-subset spatial locality within each island in a temporal manner. In L-PCN, these two methods are implemented within an **Islandization Unit**. Based on these design principles, we will now introduce the overall architecture and PCN workflow of L-PCN, with the integration of an Islandization Unit.

B. L-PCN Architecture and Workflow

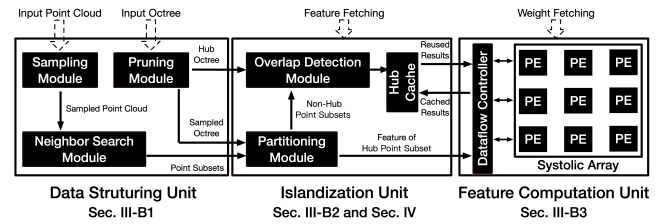


Fig. 5. Overall architecture of L-PCN

The general architecture of L-PCN is shown in Figure 5, which consists of three main units: Data Structure Unit, Islandization Unit, and Feature Computation Unit. Unlike the architecture of existing PCN accelerators (whose workflow is shown in Figure 1), the L-PCN is enhanced with an Islandization Unit after Data Structuring to exploit the spatial locality of the point cloud. In this section, we give an overview of the functions of three main units in the L-PCN architecture and their roles in the workflow. In the next section, we will describe the design of the Islandization Unit and the techniques used to exploit spatial locality.

1) *Data Structuring Unit*: The Data Structuring Unit (DSU) performs the Data Structuring step of the L-PCN workflow, which prepares the inputs for the Islandization Unit. As shown in Figure 5, there are two main inputs to the DSU: (1) the Input Point Cloud, and (2) an Input Octree, which is a spatial data structure to organize and index the input point cloud data. In L-PCN, two Islandization-based methods will utilize the Input Octree to algorithmically minimize the required computational workload in the PCN steps. The DSU consists of three main modules: Sampling Module, Neighbor Gathering Module, and Pruning Module.

Sampling Module: As shown in Figure 5, when the Input Point Cloud is processed by the DSU, it gathers all points (e.g., 1,024 in total) into multiple subsets (e.g., 512 subsets, each containing 32 points). This is accomplished by using the Sampling Module, followed by the Neighbor Search Module. The input to the Sampling Module is the Input Point Cloud. A simple example of the Sampling step for a point cloud frame is illustrated in Figure 6: The Sampling Module samples the input point cloud (black and red points) to select the central points (red). The output of the Sampling Module is the Sampled Point Cloud, containing all the central points.

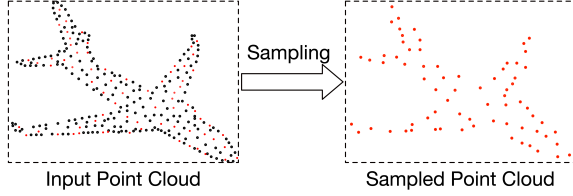


Fig. 6. Input and output of Sampling Module.

Neighbor Search Module: As in a standard PCN, after selecting the central points, the point subsets can be formed by searching the neighboring points around these central points. As shown in Figure 2(a), each point subset (black circle) is formed by searching and gathering the neighboring (black) points around a (red) central point. (In Figure 2(a), we just show some of the black circles for simplicity). This step is performed in the Neighbor Search Module, as shown in Figure 5. To calculate the distance and rank the nearest points, the Neighbor Searching step is performed based on the coordinate information $p_n = (x_n, y_n, z_n)$ of points. The method to efficiently performing the Neighbor Searching step is well developed, and can be divided into two main categories: approximate Neighbor Searching (e.g., Cresnet [14]), and accurate Neighbor Searching (e.g., PointACC [31]). In our L-PCN prototype implementations, we demonstrate that our techniques in the Islandization Unit are compatible with existing PCN accelerators, using either the approximate or accurate neighbor search methods.

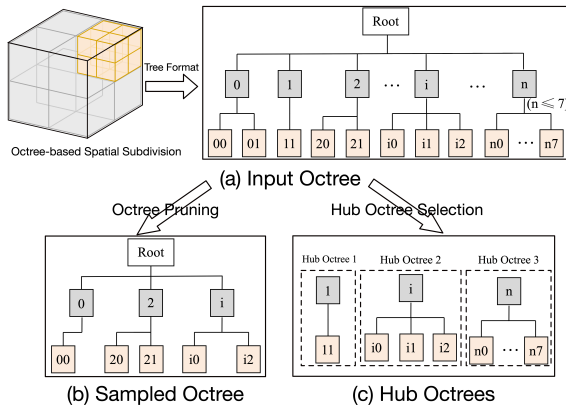


Fig. 7. Detailed operations in Pruning Module.

Pruning Module: The second input to the DSU is the Input

Octree, which is a spatial data structure to regularize the corresponding Input Point Cloud by subdividing the overall 3D space of the point cloud into voxels (i.e., cubic subdivisions) [41]. Figure 7(a) shows how an Octree subdivides the 3D space, and an example Input Octree with a depth of three (in general, Octrees can have a depth of more than three). In an Octree, each node corresponds to a non-empty voxel (i.e., containing points) and each node can have up to eight child nodes (because a voxel can be sub-divided into up to 8 sub-voxels). Using Octree search, the desired item (a point or an Octree node) can be retrieved through tree traversal, without using brute-force methods such as exhaustive search.

Like many other tree-based accelerators for Point Cloud [14], [38], [50], in L-PCN, we assume that the *Input Octree* has been built by existing methods [6]. One of the outputs from the Pruning Module is a *Sampled Octree* (Figure 7(b)), which corresponds to Sampled Point Cloud from the Sampling Module. The Sampled Octree can be efficiently obtained by pruning the Input Octree in a Pruning Module. Since the Sampled Point Cloud is made sparse from the Input Point Cloud through sampling (as shown in Figure 6), the Sampled Octree is similarly made sparse from the Input Octree by cutting off some nodes, as shown in Figure 7(b). The Sampled Octree will be used by the Octree-based Islandization method to partition point cloud, to be detailed in Section IV.

The second output from the Pruning Module is a set of *Hub Octrees*. After Sampling, L-PCN includes an extra step to pick certain number of sampled (central) points as Hub points. The point subsets centered around these Hub points are referred to as Hub point subsets, and the Octree corresponding to each Hub point subset is called its Hub Octree. Note that the Hub Octrees can be efficiently obtained by selecting some subtrees from the Input Octree, as shown in Figure 7(c). The Hub Octrees will be used by a Hub-based Scheduling algorithm in the Islandization Unit to identify the overlapping points, which will also be detailed in Section IV.

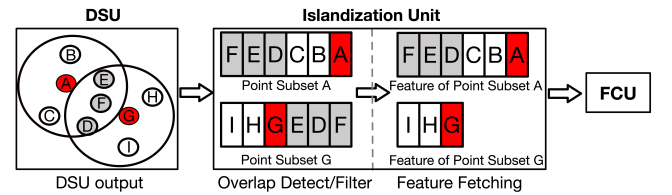


Fig. 8. L-PCN workflow with Islandization Unit. Between Point-subset A and Point-subset G (gathered by DSU), the grey points D, E, F are overlapping. The Islandization Unit can index these overlapping points and prevent them from being repeatedly fetched from memory and inputted into the FCU.

2) *Islandization Unit:* The main functions of the Islandization Unit are illustrated by the simple example in Figure 8. The Islandization Unit receives the gathered point subsets from the DSU as input, which can have a large percentage of overlapping points among neighboring point subsets. The Islandization Unit performs optimization by eliminating redundant operations through: clustering (Octree-based Islandization) and scheduling (Hub-based Scheduling) to filter

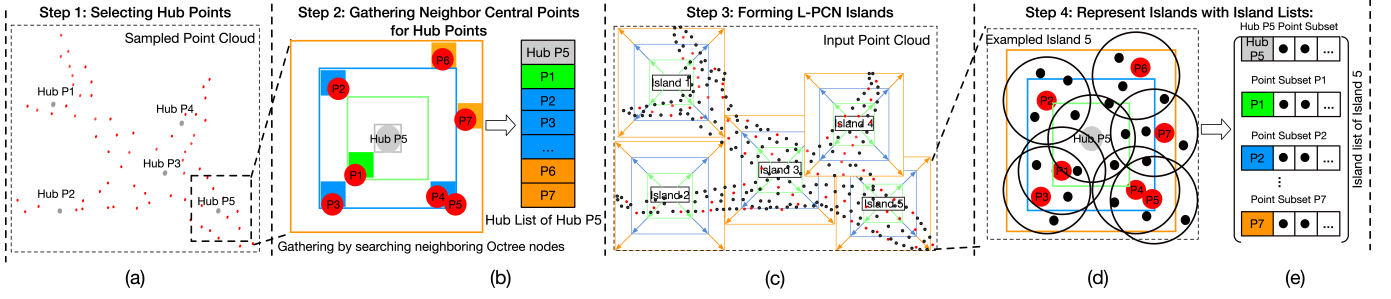


Fig. 9. (a) An example of picking five central points as Hub points (Hub P1 to Hub P5) from the Sampled Point Cloud. (b) Searching adjacent Octree nodes to gather neighboring central points for Hub P5 and create the Hub list P5. (c) Returning to the Input Point Cloud, partitioning it by forming L-PCN Islands based on the Hub Lists (d) An example of Island 5, which comprises eight point subsets. (e) The Island List used to represent the data of Island 5.

out overlapping points (as shown in Figure 8). In this way, unlike existing PCN accelerators, the features of the filtered points (e.g., overlapping points D, E, and F between Point-subset A and Point-subset G) do not need to be fetched from memory again and will not enter the downstream FCU for redundant computation. The details of the novel techniques of the Islandization Unit and their hardware support (Partitioning Module, Hub Cache, and Overlap Detection Module) will be presented in the next Section IV.

3) *Feature Computation Unit*: As shown in Figure 5, the Feature Computation Unit (FCU) in L-PCN can be built using an existing AI accelerator, combined with a Dataflow Controller, to perform the PCN’s Feature Computation step (including MLP and Pooling Layers). In general, a commercially available AI accelerator (such as the Intel NPU) is structured as a systolic array architecture. In the existing PCN accelerators (e.g., PointACC and EdgePC), the inputs to the FCU are typically provided directly by the DSU, as shown in Figure 1. On the other hand, the L-PCN architecture is enhanced with an Islandization Unit, which operates between the DSU and the FCU (as shown in Figure 5). Because the Islandization Unit prevents a large volume of overlapping points (between the point subsets gathered by DSU) from re-entering the FCU, the number of points that need to be processed by the MLP layers for L-PCN is fundamentally reduced, as compared to existing PCN accelerators.

IV. DESIGN OF ISLANDIZATION UNIT

The main role of Islandization Unit of the L-PCN is to detect the spatial locality to minimize the repeated workload of the PCN Process. The Islandization Unit uses two novel methods to exploit spatial locality: Octree-based Islandization and Hub-based Scheduling.

- As will be described in Section IV-A, the first method, **Octree-based Islandization**, is to detect spatial locality and cluster the adjacent point subsets into L-PCN Islands. The Octree-based Islandization method is supported by the **Partitioning Module** within the Islandization Unit (see Figure 5).
- As will be described in Section IV-B, the second method, **Hub-based Scheduling**, is to exploit the inter point-subset data reuse within each Island in a temporal manner.

To perform the Hub-based Scheduling, overlap detection between point subsets is required for optimizing the workload. In the Islandization Unit, overlap detection is supported by the **Overlap Detection Module** (Figure 5).

A. Octree-based Islandization

1) *Point Cloud Partitioning Method*: The Islandization Unit first performs an *Octree-based Islandization* process to gather the highly overlapping point subsets as Islands. This process is based on the Sampled Point Cloud (shown in Figure 6 which is composed of central points) and Sampled Octree (Figure 7(b)). The *Octree-based Islandization* process in L-PCN consists of four steps, as illustrated in Figure 9.

Step 1 Selecting Hub Points: From the Sampled Point Cloud, the Islandization process begins by selecting some central points as *Hub points*, as shown in Figure 9(a).

Step 2 Gathering Neighbor Central Points for Hub Points: After selecting the Hub points, a gathering process [3] is performed for each Hub point to gather its neighbor central points. This gathering process is achieved using standard Octree search operations [17] to search adjacent Octree nodes on the **Sampled Octree**. As shown in Figure 9(b), the green, blue, and orange voxels (“boxes”) correspond to the adjacent Octree nodes gathered by the first, second, and third round of the gathering process, respectively. As a result of the gathering process, each Hub point is associated with a *Hub List* which consists of this Hub point (e.g., Hub P5), together with its neighbor central points, as shown in Figure 9(b).

The process of gathering adjacent Octree nodes (and their included central points) stops when every central point belongs to a Hub List. During this process, if one Octree node is repeatedly gathered by more than one Hub List, the central point(s) within this node will be kept only in the Hub List of the nearest Hub point and removed from other Hub Lists (The node gathered in an earlier round is considered nearer). Next, we will form the L-PCN Islands based on the Hub Lists.

Step 3 Forming L-PCN Islands: After a Hub List has been created, the Islandization process is performed by returning to the original Input Point Cloud and partition it to form L-PCN Islands. An *L-PCN Island* (Island for short) is formed by gathering the point subsets (from the Input Point Cloud) whose central points are spatially neighboring and grouped into the

same Hub List. Because the overlapping elements between Hub Lists have been removed, there are no identical point subsets between Islands. Figure 9(c) shows the point cloud after Islandization, and Figure 9(d) provides a detailed view of the example Island 5. Now the point cloud is partitioned into Islands, the L-PCN will proceed with the remaining PCN steps with the granularity of Islands.

Step 4 Represent Islands with Island Lists: After the Input Point Cloud has been partitioned into Islands, each Island can be represented by an *Island List*, as shown in Figure 9(e). The Island lists will be used in the Hub-based Scheduling process to schedule the sequential execution of the rest of the PCN steps to optimize the repeated workload.

2) *Architecture Support: Partitioning Module:* The function of the Partitioning Module is to perform the Octree-based Islandization process, as explained in Section IV-A1 (and illustrated in Figure 9). The architecture of the Partitioning Module is shown in Figure 10, demonstrating the processing of an example Hub point “Hub P5” (with point subsets P1-P7). It performs the adjacent node gathering process using Octree search operations. The Octree search operations in the Partitioning Module are implemented by two **Octree-Search Engines (OSEs)** operating in parallel on a **Sampled Octree**. The function of Octree-search Engine in L-PCN is to perform Octree-search queries based on Morton code [35], using a linked-list traversal mode [33] in OSE’s Traversal Module.

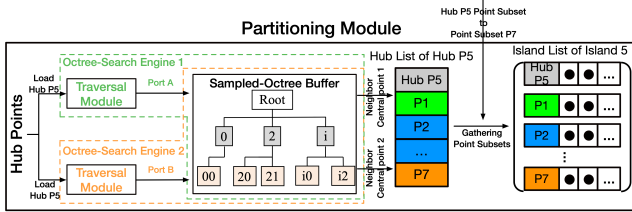


Fig. 10. Architecture of Partitioning Module.

The Partitioning Module first randomly picks a fixed number of points from sampled point cloud to serve as Hub points. As shown in Figure 10, each Hub point (e.g., Hub P5) is then inputted into the two Octree-search Engines (OSEs). The OSEs gather the neighbor central-points for this Hub point by searching the Sampled Octree. In the Partitioning Module, the Sampled Octree is stored in an **Octree Buffer** with hierarchical BRAMs in the FPGA. In our implementation, dual-port BRAM units are used to provide two parallel ports (port A and B). For this reason, we can use two OSEs to simultaneously load and process two Octree-search queries in parallel. As stated in Section IV-A1, the points found in these Octree searches are added into the corresponding Hub List until every central point has been inserted into a Hub List. At the end, each Island is formed by collecting the point subsets whose central points are included in the same Hub List.

B. Hub-based Scheduling

1) *Intra-Island Scheduling Method:* After Islandization, the L-PCN uses a Hub-based Scheduling process to exploit intra-island data reuse. This Hub-based Scheduling approach is

analogous to Cache-guided scheduling mechanisms [36], [46], ensuring that the dataflow exhibits a high degree of data locality in a temporal manner. Hub-based Scheduling leverages the data locality by dynamically caching, updating, and reusing the overlapping runtime results. As illustrated in Figure 11, for the point subsets of each Island, L-PCN first performs the complete computation for the point subset associated with the Hub point (the first row of the Island List) and caches the results into the *Hub Cache*. The reason is that the Hub point subset is the central part of an Island, sharing the most overlap with other point subsets. The Hub Cache is then progressively updated as computation proceeds within the Island.

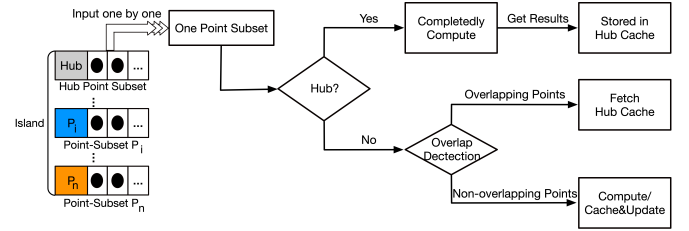


Fig. 11. Workflow of Hub-based Scheduling.

The remaining non-Hub point subsets are processed in a top-down order along the Island List, which naturally yields an inside-to-outside order within each island. As shown at the bottom of Figure 11, when processing each non-Hub point subset in this Island, L-PCN first detects overlapping points between the currently processed point subset and the points in the Hub Octree using the Octree-based *Overlap Detection* method, which will be described in Section IV-B2 (and illustrated in Figure 12). For the overlapping points, L-PCN can reuse the cached results. Only for the non-overlapping points, L-PCN has to fetch their features from memory and inputs them into the MLP layers. For these non-overlapping points, a *Tree-updating Method* is used to update them to the Hub Octree and add the corresponding results into the Hub Cache. Recall from the analysis discussed in Section II-B, the overlapping points between adjacent point subsets can reach up to around 90%, which means a significant amount of workload can be saved by L-PCN’s locality-exploiting method.

Result Delta Compensation: In modern PCNs, partial point features are often normalized relative to the central point before entering the MLP. For example, PointNet++ adjusts non-central points’ XYZ coordinates by subtracting the central point’s XYZ. Due to differences between the central points of the two point subsets, cached MLP results from one subset cannot be directly reused for another. To address this, after retrieving the cached MLP results from the Hub Cache, we compensate this result delta with an incremental adjustment derived from the difference between the central points:

For example, when processing Point-subset G (in Figure 8), we find results of N overlapping points from Point-subset A already stored in Hub Cache ($N = 3$ in this example). To reuse these three cached results, we input the Δ_{A-G} into the Feature Computation Unit and obtain the incremental adjustment $w \cdot \Delta_{A-G}$. The three fetched results are compensated by

this adjustment, following Eq. 1 [5], [16]. The compensated results are the actual values reused by Point-subset G.

$$w \cdot (P - P_G) = w \cdot ((P - P_A) + \Delta_{A-G}) = w \cdot (P - P_A) + w \cdot \Delta_{A-G} \quad (1)$$

2) *Architecture Support*: To achieve Hub-based Scheduling, L-PCN deploys an **Overlap Detection Module**, the architecture of which is shown in Figure 12. The Overlap Detection Module consists of the following main components: two **Octree-Search Engines (OSEs)**, **Hub-Octree Buffer**, and **Hub Cache**. The two OSEs and the Hub-Octree Buffer are used to identify overlapping points between the points of the previously processed point subsets of an Island, and the newly inputted point subsets. The Hub Cache is used to store the results of the potential overlapping points.

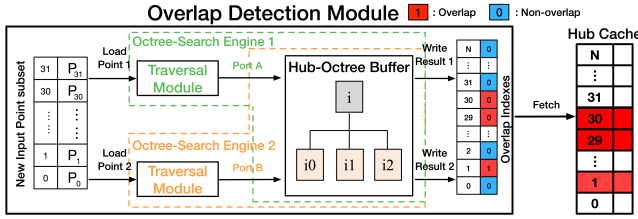


Fig. 12. Detailed architecture and workflow of the Overlap Detection Module and Hub Cache.

Overlap Detection: As stated earlier, to reuse the cached data, L-PCN needs to index the overlapping points between previously processed point subsets and newly inputted point subset. This is done in the Overlap Detection Module, where a **Hub Octree** is used to identify and index these overlapping points. A Hub Octree corresponds to the points of an Island’s Hub point subset at the beginning of processing that Island. Then during processing, the Hub Octree is dynamically updated with new non-overlapping points from the non-Hub Point Subsets.

As shown in Figure 12, the Overlap Detection Module receives a new (non-Hub) point subset as input. It employs an Octree-search method [33] to perform the overlap detection and outputs the *Overlap Indexes* (using two OSEs, similar to the Octree search in Partitioning Module). When a new non-Hub point subset arrives, the points within it are checked for overlaps by searching them in the Hub Octree (in the Hub-Octree Buffer). Points found in the Hub Octree (i.e., hits) are marked as overlapping in the Overlap Indexes, while those not found (i.e., non-hits) are marked as non-overlapping. The Overlap Indexes are used to fetch cached results (stored in the Hub Cache) for the overlapping points, and to determine which points are non-overlapping and the corresponding feature should be fetched and enter FCU.

Hub Cache: The Hub Cache is designed to store reusable results for Hub-based Scheduling. Each entry of the Hub Cache holds the MLP result of a point. Figure 12 illustrates an example architecture of the Hub Cache with N entries. The first 32 entries (assuming the size of the Hub point subset is 32) are to store the MLP results of the points from Hub point subset, while entries 33 to N store the MLP results of newly added, non-overlapping points. When a new non-Hub point subset is inputted, the Hub Cache utilizes the Overlap Indexes

(provided by the Overlap Detection Module) to return the results related to the overlapping points. During intra-island processing, the Hub Cache adopts a no-replacement policy [20] and entries are replaced only at the next island.

In summary, the Islandization Unit deploys two novel methods to detect and exploit the spatial locality of a point cloud to optimize the repeated workload of the PCN steps. The Octree-based Islandization method is used to detect spatial locality by clustering the spatially adjacent point subsets into L-PCN Islands. The Hub-based Scheduling method is used to exploit the reuse of inter point-subset data within each Island in a temporal manner. By doing so, the features of the overlapping points do not need to be fetched from memory again and will not re-enter the downstream FCU for redundant computation.

V. OVERALL DATAFLOW AND OPTIMIZATION OF L-PCN

In the previous section (Section IV), we presented the design of L-PCN’s Islandization Unit and its methods for detecting and leveraging spatial locality. In this section, we will provide the details of the dataflow of the overall L-PCN architecture to show how the optimization strategy takes advantage of the data reuse methods implemented in the Islandization Unit.

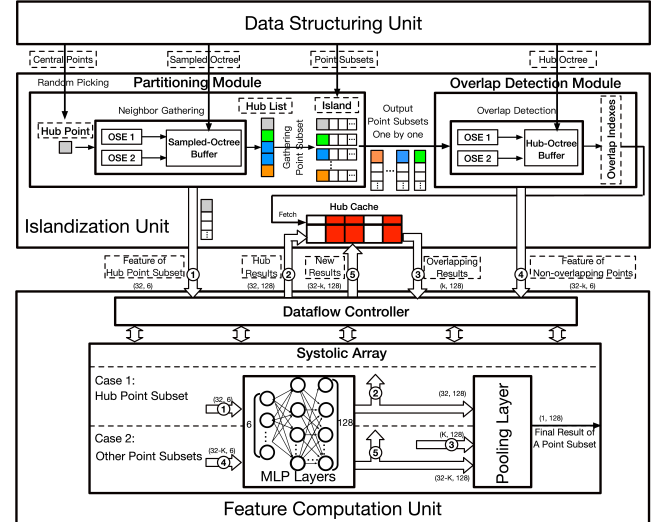


Fig. 13. L-PCN architecture with active dataflow.

Example: L-PCN’s Dataflow to Exploit Spatial Locality: Figure 13 illustrates the overall L-PCN architecture (a detailed version of the architecture presented in Figure 5), showcasing the active dataflow using examples from Figures 10 and 12. As shown in Figure 13, the Islandization Unit consists of a Partitioning Module, an Overlap Detection Module, and an attached Hub Cache. It receives its inputs from the Data Structuring Unit and outputs data to the Feature Computation Unit (FCU). Within the FCU in Figure 13, the arrows represent dataflows, while the MLP and Pooling Layers are performed by a Systolic Array module (e.g., NPU). In Figure 13, to support the Islandization Unit in exploiting spatial locality, the FCU dataflow is divided into two cases:

- Case 1 (top section) illustrates the dataflow for the Hub Point Subset. This involves completing computations

(arrows ①) for all 32 of its points and subsequently caching the 32 results (arrows ②) into Hub Cache.

- Case 2 (bottom section) illustrates the dataflow for the remaining non-Hub point subsets based on Hub-based Scheduling. This dataflow is further detailed in Figure 14. The Overlap Detection Module divides the 32 points within each non-Hub point subset into two groups: K overlapping and $(32 - K)$ non-overlapping points. For the K overlapping points, the Islandization Unit retrieves the cached MLP results (with dimensions of $(K, 128)$) from the Hub Cache (arrow ③ in Figure 13) and inputs them into the Pooling Layer. For the $(32 - K)$ non-overlapping points, the Islandization Unit input them into the MLP layer (arrow ④) from the Overlap Detection Module. The resulting MLP outputs (with dimensions of $(32 - K, 128)$) are then forwarded to the Pooling Layer and stored in the available entries in the Hub Cache (arrow ⑤). These points are subsequently updated in the Hub Octree.

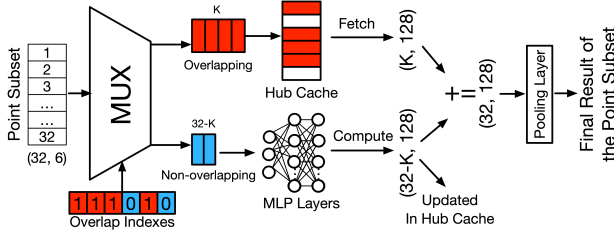


Fig. 14. Example of Data Reusing Method with overlap detection. In this example, the feature dimension of a point subset is changed from $(32, 6)$ to $(32, 128)$ during MLP layers. K is the number of detected overlapping points.

In summary, due to the optimization provided by the L-PCN’s Islandization Unit, the workload of the FCU’s MLP layers, which accounts for over 98% of the computational workload in the FC step, is fundamentally reduced. Within an island, compared to the traditional method, only the workload for Hub point subset remains unchanged. For the remaining non-Hub point subsets, the $K/32$ overlapping points can avoid repeated feature fetching and re-entering the MLP layers. As we will demonstrate in Section VI, this leads to significant workload savings for the overall PCN workflow.

VI. EVALUATION

A. Evaluation Setup

Benchmarks To demonstrate the optimization achieved by our methods, we evaluate the prototype L-PCN implementations using two representative PCN models, PointNet++ and DGCNN, on four point cloud datasets, as listed in Table I. These benchmarks represent point cloud datasets of different sizes and diverse application scenarios.

TABLE I
EVALUATION BENCHMARKS

Dataset	Input Point#	Model	Task Type
ModelNet40	1024	PointNet++(c) [39]	Classification
ShapeNet	2048	PointNet++(ps) [39]	Part Segmentation
S3DIS	4096	PointNet++(s) [39]	Semantic Segmentation
ModelNet40	1024	DGCNN(c) [48]	Classification
ScanNet	8192	DGCNN(s) [48]	Semantic Segmentation

Baselines. Four existing PCN accelerators have been selected as our baselines for comparison. Two of them—PointACC [31] and HgPCN [19]—employ accurate Data Structuring methods, while the other two—EdgePC [53] and Crescent [14]—utilize approximate Data Structuring methods. Most existing PCN accelerators, including the four baselines, adopt a standard architecture that performs data structuring with a customized Data Structuring Unit (DSU) and delegates feature computation to a Feature Computation Unit (FCU). For each baseline, we customized their DSUs following the methods described in their respective original works. For consistency, all baselines deploy a 16×16 systolic-array FCU.

Finally, two additional baseline accelerators are GDPCA [5] and Mesorasi [16]. Both incorporate workload-reduction mechanisms, *Geometry-aware Differential Update* (in GDPCA) and *Delay-Aggregation* (in Mesorasi), to reduce the workload of feature computation. We compare the feature computation speedup of L-PCN, GDPCA, and Mesorasi.

Prototype L-PCN Implementations All the prototypes are implemented on an Intel Arria 10 GX FPGA board using SystemVerilog and VHDL, operating at 250 MHz. There are eight implementations: four baseline accelerators (following their respective methods as described above), and four corresponding L-PCN prototypes (same four baseline accelerators, with the addition of the Islandization Unit). The default configuration of Islandization Unit is that each Island includes 32 subsets; the Hub-Cache capacity is sized to $2 \times$ the maximum feature size of one subset. Thus, when comparing an L-PCN prototype to a baseline accelerator, the L-PCN’s DSU employs the same Data Structuring (DS) method as used by that baseline. For example, when comparing L-PCN to PointACC, the DSU in the corresponding L-PCN prototype adopts the method used in “Mapping Unit” from PointACC. Similarly, the FCU of the L-PCN prototype is also a 16×16 systolic array, as in PointACC. In this manner, we can showcase the performance improvement of an existing PCN accelerator when it is enhanced with the Islandization Unit as a plug-in.

B. Theoretical Workload Optimization

Before we present the measured performance of the prototype implementations on the FPGA, we first present a software-based analysis to quantify the advantage of our method for exploiting spatial locality. The results of theoretical optimization against the traditional method (without data reuse) are shown in Figure 15.

Memory-fetch Saving: As previously stated, once point subsets are formed, the late stage of data structuring requires fetching the feature information of these points from memory to prepare inputs for feature computation. The Islandization Unit reduces memory fetches by eliminating repeated fetches for overlapping points. In Figure 15, the first (green) bars represent the optimization ratio for feature fetching, while the second (yellow) bars show the impact of reduced feature fetching on overall memory access (including both weight and feature fetching). As shown in Figure 15, for the benchmark PCN tasks in Table I, L-PCN’s methods for exploiting spatial

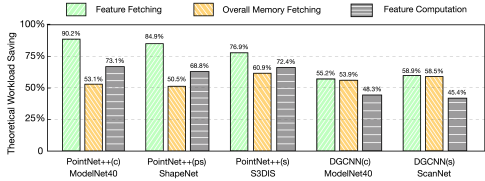


Fig. 15. Theoretical workload optimization

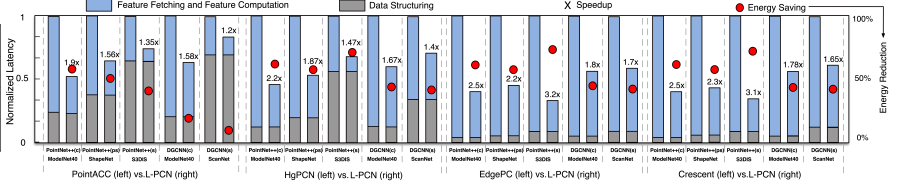


Fig. 16. Performance comparison between L-PCN prototypes and baselines.

locality reduce feature fetches ranging from 55.2% to 90.2% and overall memory access from 50.5% to 60.9%.

Computation Saving: For Feature Computation, L-PCN prevents overlapping points from re-entering the MLP layers. As shown in the third (grey) bars of Figure 15, for the benchmark PCN tasks, L-PCN’s methods for exploiting spatial locality reduce computational workload ranging from 45.4% to 73.1%. The savings in feature computation are slightly less than those in point-feature fetching primarily due to the one-time overhead of supplementary computation introduced by result delta compensation, as discussed at the end of Section IV-B1. Despite this, our techniques for exploiting spatial locality still achieve substantial reductions in computational workload.

C. Comparison with Baseline Accelerators

In this section, we present the evaluation results by comparing the baseline accelerators with L-PCN prototypes. As mentioned, for the first four baseline PCN accelerators (two accurate and two approximate Data Structuring methods), each L-PCN prototype utilizes the same DSU and FCU structure as its corresponding baseline. The key difference is the integration of an Islandization Unit between the DSU and FCU. By doing so, this comparison showcases the performance improvements achieved by enhancing baseline accelerators with L-PCN’s Islandization Unit.

Figure 16 illustrates the performance improvements of L-PCN prototypes over the four baselines, using benchmarks in Table I. For each benchmark, the left bar represents the latency breakdown of the baseline accelerator (end-to-end latency normalized to 1), while the right bar represents the normalized latency of the corresponding L-PCN prototype along with its speedup (number, from cycle-accurate RTL-level simulation) and energy reduction (red dot, estimated from activity factors tracing). In all benchmarks, the latency overhead of the Islandization Unit is less than 1% and is therefore omitted from Figure 16 (cycle-accurate latency will be reported in Section VI-G).

The first two PCN accelerators—PointACC and HgPCN—follow the traditional Data Structuring approach, performing accurate neighbor gathering. PointACC accelerates the neighbor gathering step using a hardware-based ranking kernel; whereas HgPCN utilizes an Octree method to narrow the neighbor gathering space and then conducts the ranking-based gathering. As shown in Figure 16, L-PCN prototypes can achieve speedups ranging from 1.2× to 1.9× and 38% average energy reduction over PointACC, and from 1.4× to 2.2× and 55% average energy reduction over HgPCN.

The second two PCN accelerators—EdgePC and Crescent—leverage the approximate nature of DNN algorithms to enable efficient approximate neighbor search for data structuring. EdgePC utilizes Morton code to structure points and performs approximate neighbor gathering through an index-based method, while Crescent employs a KD-tree method to perform approximate neighbor gathering by efficient KD-tree search. As shown in Figure 16, when compared to these two approximate PCN accelerators, L-PCN prototypes achieve speedups ranging from 1.7× to 3.2× and 56% average energy reduction over EdgePC, and speedups from 1.65× to 3.1× and 56% average energy reduction over Crescent.

L-PCN vs. GDPCA: We further compare L-PCN with GDPCA [5] and Mesorasi [16], two existing PCN accelerators that incorporate workload-reduction mechanisms for feature computation. We first present the comparison with GDPCA, which proposes a *Geometry-aware Differential Update* method to leverage the inter-point feature similarity and perform computation on inter-point feature delta [34], which can reduce the input bit width and enable speedup by using the classic Bit-Pragmatic accelerator (PRA) [1]. However, for GDPCA, the total number of inputs fed into the MLP layers remains unreduced, as in the common method. Consequently, the benefit of bits reduction is limited when translated into actual speedup. In Figure 17, we compare the feature computation speedup provided by GDPCA (the 1st bar of each group) and L-PCN (the 2nd bar) against the commonly used method (normalized as 1, without workload reduction). Compared to GDPCA’s method, our mechanism for exploiting spatial locality prevents overlapping points from re-entering the MLP, fundamentally reducing the total number of feature computations.

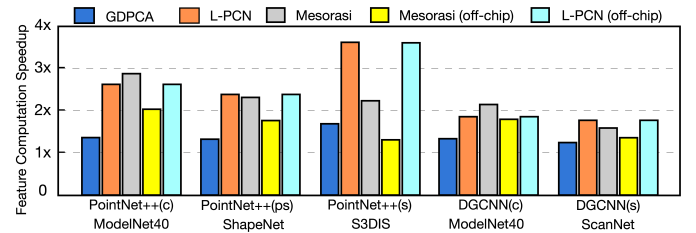


Fig. 17. Feature Computation speedup of GDPCA, L-PCN, and Mesorasi.

L-PCN vs. Mesorasi: Mesorasi proposes the *Delayed-Aggregation* to reduce feature computation workload. Mesorasi first applies MLP to all points twice to generate two sets of features: $MLP(X, Y, Z, f_1, \dots, f_n)$ and $MLP(X, Y, Z, 0, \dots, 0)$, and stores them in a Point Feature Table (PFT). For

each 32-point subset, it retrieves from PFT the 32 features $MLP(X, Y, Z, f_1, \dots, f_n)$ and one central-point feature $MLP(X_c, Y_c, Z_c, 0, \dots, 0)$, then concatenates them into the 32 $MLP(X - X_c, Y - Y_c, Z - Z_c, f_1, \dots, f_n)$ results of this subset.

Although Mesorasi achieves higher MLP computation reduction compared to L-PCN, its Delayed-Aggregation method essentially shifts the burden to the PFT [54]. The intensive memory access of Delayed-Aggregation phase makes it the new bottleneck [5]. Moreover, the separation of MLP (which performs computation) and Delayed-Aggregation (which involves memory access from PFT) into two distinct serialized phases prevents the overlap of computation and memory fetching. This limitation worsens when the PFT exceeds the capacity of on-chip memory, making off-chip memory access the critical bottleneck. Figure 17 analyzes the feature computation speedup of L-PCN and Mesorasi under two settings: (1) when intermediate data is stored in on-chip buffer (the 2nd and 3rd bars of each group), and (2) when it is stored in off-chip memory (the 4th and 5th bars). Under the on-chip setting, L-PCN still achieves higher speedup in most benchmarks. Under the off-chip setting, L-PCN largely maintains its speedup through overlapping memory access and computation, while Mesorasi incurs obvious performance degradation due to its inability to hide off-chip memory access latency.

Limitation Discussion: L-PCN’s optimization primarily relies on overlaps between subsets and thus shares a common limitation with Mesorasi: the inability to optimize certain no-overlap PCN layers. For instance, in PointNet++ (c), the last Set Abstraction which contains only one subset and the final classification layers (together account for $\sim 11\%$ of feature computation FLOPs) cannot benefit from our method. Similarly, for PointNeXt (a large-scale PCN, evaluated next), the beginning Stem MLP that expands per-point features ($\sim 0.1\%$ of FLOPs) and the segmentation decoder ($\sim 3\%$ of FLOPs) remain unoptimized. Nevertheless, their workload is low relative to the remaining “layers with overlap” and has limited impact on overall gains.

D. Additional Baseline and Benchmark for Large-Scale PCNs

In addition to the benchmarks in Table I, we further evaluate the performance of L-PCN for another scenario: large-scale point cloud processing. PointNeXt [40] and PointVector [12] are scalability-oriented variants of PointNet++ for large-scale point cloud inputs. To the best of our knowledge, FractalCloud [18] is the only accelerator designed for PointNeXt and PointVector. We therefore include it as a baseline and adopt its original benchmarks: the PointNeXt-S and PointVector-L models on the S3DIS dataset with input points ranging from 8K to 289K. Similar to the previous evaluation, we first analyze L-PCN’s theoretical workload optimization for PointNeXt and PointVector, and then evaluate the performance gain when L-PCN’s Islandization Unit is integrated as a plug-in into FractalCloud. As shown in Figure 18, for these two models, L-PCN’s methods for exploiting spatial locality reduce feature fetching by 66.4%–93.8% (translating to 60.9%–93.6% overall memory access savings) and reduce computational

workload by 53.2%–80.6%. In general, L-PCN shows even higher optimization on larger-scale point clouds, as higher point density leads to greater overlap among subsets.

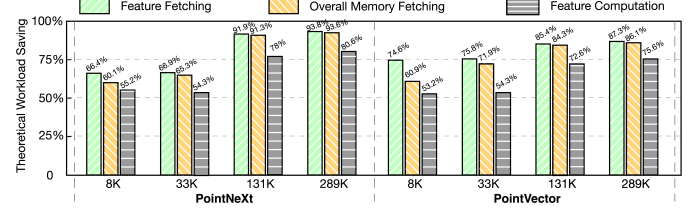


Fig. 18. Theoretical workload optimization for PointNeXt and PointVector.

FractalCloud proposes a block-based (approximate) data structuring to avoid prohibitive global-search overhead for large point clouds and enable block-wise parallelism. The original FractalCloud deploys Mesorasi’s Delayed-Aggregation for feature computation optimization. Our L-PCN prototype adopts the same data structuring method but replaces Delayed-Aggregation with the Islandization Unit and compares with vanilla FractalCloud. As shown in Figure 19, L-PCN achieves speedups of $1.2\times$ – $2.1\times$ and an average energy saving of 48.5% over FractalCloud. This improvement stems from the additional feature-computation optimization of the Islandization Unit over Delayed-Aggregation (analyzed in Figure 17).

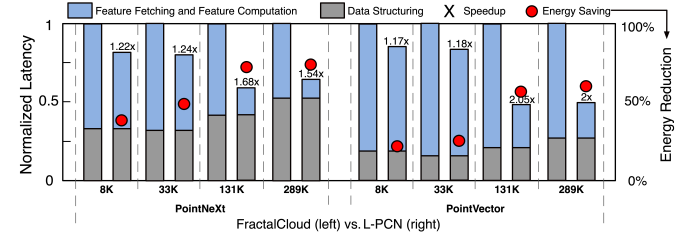


Fig. 19. Performance Comparison of L-PCN prototype and FractalCloud.

E. Accuracy Comparison

Even though L-PCN’s methods reuse MLP results between adjacent point subsets, the result delta compensation (Section IV-B1) introduces potential accuracy degradation. That is because MLPs contain non-linear activation layers, and $MLP(A - B) \approx MLP(A) - MLP(B)$ only holds approximately (e.g., Mesorasi can incur up to 0.9% accuracy loss).

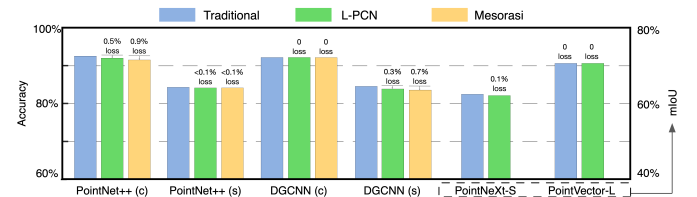


Fig. 20. Accuracy comparison among traditional method, L-PCN, and Mesorasi across different benchmarks.

Figure 20 analyzes the resulting accuracy loss of L-PCN and Mesorasi relative to traditional methods on benchmarks used in Mesorasi [16] and on PointNeXt and PointVector,

showing that L-PCN better maintains accuracy. This is mainly because Mesorasi applies approximation to all MLP results, whereas L-PCN selectively applies approximation (by result delta compensation) to MLP results related to “overlapping” points and preserves exact computation for “non-overlapping” points [43]. Moreover, these non-overlapping points often play a more critical role in prediction, as they tend to lie near the shape boundaries of the point cloud (see Figure 21). The MLP results of these boundary points are typically higher and more likely to be retained by max pooling, while the approximated results tend to be pooled out. In particular, DGCNN (c) and PointVector-L exemplify cases where accuracy loss can be avoided when activation is applied only at the end of each building block. This allows the reuse of pre-activation results to fully compensate for the result delta, since $CONV(A-B) = CONV(A) - CONV(B)$ holds exactly.

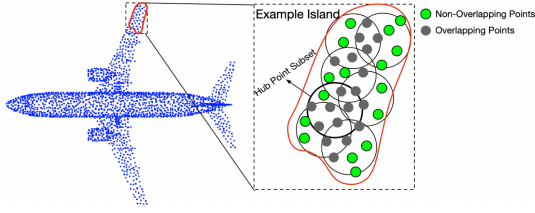


Fig. 21. Empirical analysis shows that non-overlapping (usually boundary) points more effectively capture the shape of the “airplane wings”.

F. Sensitivity Study

Figure 22 examines how the Islandization Unit’s two hyperparameters, **Island size** (algorithmic) and **Hub Cache capacity** (resource-related), affect performance and accuracy on PointNet++(c) benchmark. Default-configuration results were previously reported in Figures 15-20. Based on the sensitivity study in Figure 22, smaller Island sizes generally improve theoretical workload reduction and latency/energy efficiency. For Hub Cache capacity, larger caches show negligible impact for small islands (indicating overprovisioning) but improve performance for large islands. Since L-PCN preserves precise computation for non-overlapping points, configurations with less data reuse generally exhibit higher accuracy. For example, comparing the 4th and 6th groups of bars shows that a larger Island Size leads to less data reuse (the 1st and 2nd bars of each group), decreasing speedup (the 3rd bars) from $2.94\times$ to $1.83\times$ but improving accuracy (the top numbers) by 0.2%. Moreover, such adjustable accuracy for different demands is not supported by Mesorasi’s fully approximated method.

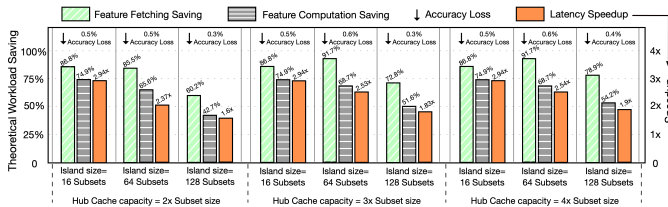


Fig. 22. Sensitivity Study of Islandization hyperparameters

G. Detailed Configuration of the L-PCN Prototype

Table II reports the resource utilization and cycle-accurate latency of an L-PCN prototype for PointNet++(c). This prototype adopts the “Mapping Unit” design from PointACC for its Data Structuring Unit, which consists primarily of 16 parallel distance calculators and a 32-way bitonic sorter, and deploys a 16×16 systolic array as its Feature Computation Unit. Its Islandization Unit (default hyperparameter setting) mainly comprises the Partitioning and Overlap Detection modules (implemented with two pipelined Octree-Search Engines), along with an attached BRAM-based Hub Cache. Overall, the Islandization Unit introduces acceptable resource overhead, but provides significant improvement as demonstrated.

TABLE II
RESOURCE UTILIZATION AND CYCLE-ACCURATE LATENCY

Resource Type	DSU Usage	Islandization Unit Usage	FCU Usage	Total Resource on device
Logic (ALMs)	26,071	14,361	6,229	119,900
Register	19,389	10,140	12,049	239,800
DSP	48	0	256	984
BRAM	278,528	770,048	9,437,184	18,247,680
Latency (cycles)	1,046,461	1,497	1,263,176	–

Based on the FPGA prototype in Table II, we implement the L-PCN ASIC version, operating at 1 GHz. We synthesize it using Synopsys Design Compiler under TSMC 28nm technology, with SRAM generated by the TSMC memory Compiler. We use PrimeTime PX to obtain power measurements based on switching activities. Figure 23 shows the unit-wise area and power breakdown of the L-PCN prototype. For each unit, white-shaded portions indicate SRAM-related components (area or power), while black-shaded portions indicate logic-related components. Overall, the Islandization Unit incurs $\sim 14\%$ area overhead and $\sim 10\%$ power overhead.

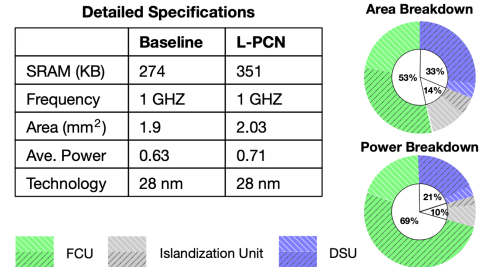


Fig. 23. Detailed specifications and area/power breakdown of the L-PCN ASIC prototype.

VII. RELATED WORK

PCN accelerator. The compelling performance of PCNs has driven the development of domain-specific PCN designs. Prior work on PCN acceleration primarily spans customized Data Structuring units [14], [18], [19], [31], [53], [55], Processing-in-Memory [9], and ISA-level extensions [23]. However, these works inevitably incur redundant operations during the Feature Computation step due to cross-subset overlapping points. Both Mesorasi [16] and our L-PCN target redundancy mitigation in

feature computation, but differ in their data-reuse mechanisms and precision (as detailed in Sections VI-C and VI-E, respectively). For data reuse, Mesorasi adopts a “precomputation-and-refetch” scheme. This method requires a large buffer to store all precomputed data and the refetch latency cannot be hidden, making performance limited by memory bandwidth. Alternatively, L-PCN employs a runtime reuse mechanism that effectively hides memory-access latency. For precision, unlike Mesorasi’s fully approximated method, L-PCN is analogous to selective precision [43]. This allows it to better preserve accuracy and provide higher cross-domain robustness.

Point Cloud Partitioning. Input partitioning is a common optimization technique in graph processing [7], [20], [51] and has recently been applied to point cloud domain. GDPCA [5] leverages Space-Uniform Partitioning to cluster points with similar features and performs computation on inter-point feature deltas. SimDiff [29], an extended version of GDPCA, introduces Density-Uniform Partitioning to better balance the workload across partitions. For efficient large-scale point cloud partitioning, FractalCloud proposes a lightweight Shape-Uniform Partitioning scheme that skips most point traversals. This partitioning constrains the Data-Structuring search space to intra-block regions, thereby avoiding global-search overhead and enabling block-wise parallelism. Unlike prior approaches, L-PCN performs partitioning by clustering neighboring central points to organize their associated point subsets into Islands, enabling cross-subset data reuse.

VIII. CONCLUSION AND BROADER APPLICATIONS

The development of 3D sensors has driven the advancement of PCNs and the domain-specific PCN accelerators. Currently, most existing PCN accelerators primarily target optimizing the PCN-specific Data Structuring step. In this work, we observe that adjacent point subsets formed during Data Structuring step exhibit significant spatial overlap, leading to data redundancy in the subsequent PCN workflow. To leverage this characteristic, we introduce L-PCN, which further accelerates PCNs by exploiting the spatial locality. L-PCN enhances the PCN workflow by incorporating an Islandization Unit, which comprises two key techniques: Octree-based Islandization, which detects spatial locality by partitioning the point cloud at runtime and clustering highly overlapping subsets into islands; and Hub-based Scheduling, which exploits spatial locality through intra-island data reuse. Experiments show that L-PCN reduces memory access for feature fetching by 55.2%–93.8% and feature computation workload by 45.4%–80.6%. L-PCN also outperforms state-of-the-art PCN accelerators (GDPCA and Mesorasi) in workload reduction. Moreover, the Islandization Unit can be seamlessly integrated into existing PCN accelerators, enhancing their performance by $1.2\times$ – $3.2\times$.

Broader Applications: Emerging 3D data (e.g., point clouds, 3D Gaussians) are fundamental to modern vision domains like autonomous driving and AR/VR, yet their spatial discreteness and sparsity pose unique challenges for exploiting locality. Our Islandization-based methods address this as a key contribution. Although our primary benchmarks are point-based PCNs, our

methods can be extended to other 3D workloads, such as sparse-convolution PCNs [21] and Gaussian-based rendering [15], [24], [30].

IX. ACKNOWLEDGEMENTS

This work was supported by the I/UCRC Program of the National Science Foundation under Grant Number 1738420.

REFERENCES

- [1] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O’Leary, R. Genov, and A. Moshovos, “Bit-pragmatic deep neural network computing,” in *Proceedings of the 50th annual IEEE/ACM international symposium on microarchitecture*, 2017, pp. 382–394.
- [2] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall, “Semantickitti: A dataset for semantic scene understanding of lidar sequences,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9297–9307.
- [3] J. Behley, V. Steinhage, and A. B. Cremers, “Efficient radius neighbor search in three-dimensional point clouds,” in *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2015, pp. 3625–3630.
- [4] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su *et al.*, “Shapenet: An information-rich 3d model repository,” *arXiv preprint arXiv:1512.03012*, 2015.
- [5] C. Chen, X. Zou, H. Shao, Y. Li, and K. Li, “Point cloud acceleration by exploiting geometric similarity,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1135–1147.
- [6] F. Chen, R. Ying, J. Xue, F. Wen, and P. Liu, “Parallelnn: A parallel octree-based nearest neighbor search accelerator for 3d point clouds,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 403–414.
- [7] X. Chen, Y. Wang, X. Xie, X. Hu, A. Basak, L. Liang, M. Yan, L. Deng, Y. Ding, Z. Du *et al.*, “Rubik: A hierarchical architecture for efficient graph learning,” *arXiv preprint arXiv:2009.12495*, 2020.
- [8] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, “Multi-view 3d object detection network for autonomous driving,” in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2017, pp. 1907–1915.
- [9] X.-J. Chen, H.-P. Chen, and C.-L. Yang, “Pointcim: A computing-in-memory architecture for accelerating deep point cloud analytics,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1309–1322.
- [10] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.
- [11] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner, “ScanNet: Richly-annotated 3d reconstructions of indoor scenes,” in *Proc. Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2017.
- [12] X. Deng, W. Zhang, Q. Ding, and X. Zhang, “Pointvector: a vector representation in point cloud analysis,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 9455–9465.
- [13] H. Esmaeilzadeh, A. Sampson, L. Ceze, and D. Burger, “Neural acceleration for general-purpose approximate programs,” *Communications of the ACM*, vol. 58, no. 1, pp. 105–115, 2014.
- [14] Y. Feng, G. Hammonds, Y. Gan, and Y. Zhu, “Crescent: taming memory irregularities for accelerating deep point cloud analytics,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 962–977.
- [15] Y. Feng, W. Lin, Y. Cheng, Z. Liu, J. Leng, M. Guo, C. Chen, S. Sun, and Y. Zhu, “Lumina: Real-time mobile neural rendering by exploiting computational redundancy,” *arXiv preprint arXiv:2506.05682*, 2025.
- [16] Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu, “Mesorasi: Architecture support for point cloud analytics via delayed-aggregation,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1037–1050.
- [17] S. F. Frisken and R. N. Perry, “Simple and efficient traversal methods for quadtrees and octrees,” *Journal of Graphics Tools*, vol. 7, no. 3, pp. 1–11, 2002.

- [18] Y. Fu, C. Zhou, H. Ye, B. Duan, Q. Huang, C. Wei, C. Guo, H. Li, Y. Chen *et al.*, “Fractalcloud: A fractal-inspired architecture for efficient large-scale point cloud processing,” *arXiv preprint arXiv:2511.07665*, 2025.
- [19] Y. Gao, C. Jiang, W. Piard, X. Chen, B. Patel, and H. Lam, “Hgcpcn: A heterogeneous architecture for e2e embedded point cloud inference,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1588–1600.
- [20] T. Geng, C. Wu, Y. Zhang, C. Tan, C. Xie, H. You, M. Herboldt, Y. Lin, and A. Li, “I-gcn: A graph convolutional network accelerator with runtime locality enhancement through islandization,” in *MICRO-54: 54th annual IEEE/ACM international symposium on microarchitecture*, 2021, pp. 1051–1063.
- [21] B. Graham, M. Engelcke, and L. Van Der Maaten, “3d semantic segmentation with submanifold sparse convolutional networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 9224–9232.
- [22] Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Bennamoun, “Deep learning for 3d point clouds: A survey,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, no. 12, pp. 4338–4364, 2020.
- [23] M. Han, L. Wang, L. Xiao, H. Zhang, B. Jiang, X. Xie, J. Zhu, S. Wei, and L. Liu, “Pointisa: Isa-extensions for efficient point cloud analytics via architecture and algorithm co-design,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 1867–1881.
- [24] X. Huang, H. Zhu, T. Ma, Y. Xiong, F. Liu, Z. He, Y. Gan, Z. Liu, J. Leng, Y. Feng *et al.*, “Splatonic: Architectural support for 3d gaussian splatting slam via sparse processing,” in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–14.
- [25] L. Kästner, V. C. Frasinianu, and J. Lambrecht, “A 3d-deep-learning-based augmented reality calibration method for robotic environments using depth sensor data,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 1135–1141.
- [26] H. Kwon, A. Samajdar, and T. Krishna, “Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects,” *ACM Sigplan Notices*, vol. 53, no. 2, pp. 461–475, 2018.
- [27] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “Pointpillars: Fast encoders for object detection from point clouds,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 12 697–12 705.
- [28] T. Le and Y. Duan, “Pointgrid: A deep network for 3d shape understanding,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 9204–9214.
- [29] Y. Li, M. Li, C. Chen, X. Zou, H. Shao, F. Tang, and K. Li, “Simdiff: Point cloud acceleration by utilizing spatial similarity and differential execution,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 2, pp. 568–581, 2024.
- [30] W. Lin, Y. Feng, and Y. Zhu, “Metasapiens: Real-time neural rendering with efficiency-aware pruning and accelerated foveated rendering,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 669–682.
- [31] Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han, “Pointacc: Efficient point cloud accelerator,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 449–461.
- [32] L. Liu, B. Su, J. Jiang, G. Wu, C. Guo, C. Xu, and H. F. Yang, “Towards accurate and efficient 3d object detection for autonomous driving: A mixture of experts computing system on edge,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 25 903–25 913.
- [33] D. Madeira, A. Montenegro, E. Clua, and T. Lewiner, “Gpu octrees and optimized search,” in *Proceedings of VIII Brazilian Symposium on Games and Digital Entertainment*, 2009, pp. 73–76.
- [34] M. Mahmoud, K. Siu, and A. Moshovos, “Diffy: A déjà vu-free differential deep neural network accelerator,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 134–147.
- [35] G. M. Morton, “A computer oriented geodetic data base and a new technique in file sequencing,” 1966.
- [36] A. Mukkara, N. Beckmann, M. Abeydeera, X. Ma, and D. Sanchez, “Exploiting locality in graph analytics through hardware-accelerated traversal scheduling,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 1–14.
- [37] A. Mukkara, N. Beckmann, and D. Sanchez, “Cache-guided scheduling: Exploiting caches to maximize locality in graph processing,” *AGP’17*, 2017.
- [38] R. Pinkham, S. Zeng, and Z. Zhang, “Quicknn: Memory and performance optimization of kd tree based nearest neighbor search for 3d point clouds,” in *2020 IEEE International symposium on high performance computer architecture (HPCA)*. IEEE, 2020, pp. 180–192.
- [39] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” *Advances in neural information processing systems*, vol. 30, 2017.
- [40] G. Qian, Y. Li, H. Peng, J. Mai, H. Hammoud, M. Elhoseiny, and B. Ghanem, “Pointnext: Revisiting pointnet++ with improved training and scaling strategies,” *Advances in neural information processing systems*, vol. 35, pp. 23 192–23 204, 2022.
- [41] B. Schön, A. S. M. Mosa, D. F. Lafer, and M. Bertolotto, “Octree-based indexing for 3d pointclouds within an oracle spatial dbms,” *Computers & Geosciences*, vol. 51, pp. 430–438, 2013.
- [42] A. Sinha, J. Bai, and K. Ramani, “Deep learning 3d shape surfaces using geometry images,” in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VI 14*. Springer, 2016, pp. 223–240.
- [43] Z. Song, B. Fu, F. Wu, Z. Jiang, L. Jiang, N. Jing, and X. Liang, “Drq: dynamic region-based quantization for deep neural network acceleration,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 1010–1021.
- [44] J. D. Stets, Y. Sun, W. Corning, and S. W. Greenwald, “Visualization and labeling of point clouds in virtual reality,” in *SIGGRAPH Asia 2017 Posters*, 2017, pp. 1–2.
- [45] J. Sun, Q. Zhang, B. Kailkhura, Z. Yu, C. Xiao, and Z. M. Mao, “Modelnet40-c: Arobustness benchmark for 3d point cloud recognition under corruption.”
- [46] D. Tripathy, A. Abdolrashidi, L. N. Bhuyan, L. Zhou, and D. Wong, “Paver: Locality graph-based thread block scheduling for gpus,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 18, no. 3, pp. 1–26, 2021.
- [47] C. Wang, B. Samari, and K. Siddiqi, “Local spectral graph convolution for point set feature learning,” in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 52–66.
- [48] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph cnn for learning on point clouds,” *Acm Transactions On Graphics (tog)*, vol. 38, no. 5, pp. 1–12, 2019.
- [49] Y. Wang, S. Zhang, B. Wan, W. He, and X. Bai, “Point cloud and visual feature-based tracking method for an augmented reality-aided mechanical assembly system,” *The International Journal of Advanced Manufacturing Technology*, vol. 99, no. 9, pp. 2341–2352, 2018.
- [50] T. Xu, B. Tian, and Y. Zhu, “Tigris: Architecture and algorithms for 3d perception in point clouds,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 629–642.
- [51] J. Yan, H. Ito, Y. Nagahara, K. Kawamura, M. Motomura, T. Van Chu, and D. Fujiki, “Bingogcn: Towards scalable and efficient gnn acceleration with fine-grained partitioning and slt,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1910–1924.
- [52] B. Yang, W. Luo, and R. Urtasun, “Pixor: Real-time 3d object detection from point clouds,” in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2018, pp. 7652–7660.
- [53] Z. Ying, S. Bhuyan, Y. Kang, Y. Zhang, M. T. Kandemir, and C. R. Das, “Edgepc: Efficient deep learning analytics for point clouds on edge devices,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.
- [54] C. Zhou, Y. Fu, M. Liu, S. Qiu, G. Li, Y. He, and H. Jiao, “An energy-efficient 3d point cloud neural network accelerator with efficient filter pruning, mlp fusion, and dual-stream sampling,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [55] C. Zhou, T. Huang, Y. Ma, Y. Fu, X. Song, S. Qiu, J. Sun, M. Liu, G. Li, Y. He *et al.*, “23.4 nebula: A 28nm 109.8 tops/w 3d pnn accelerator featuring adaptive partition, multi-skipping, and block-wise aggregation,” in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 68. IEEE, 2025, pp. 412–414.